CHATER 1

INTRODUCTION

1.1 LIGHT BOUNDARY SYSTEM

In modern cricket, technology plays a vital role in ensuring accurate decisions and enhancing the viewing experience. One critical aspect is the boundary line, where decisions regarding whether a ball has crossed it can significantly influence the game's outcome. The Light Boundary System is a simple yet effective innovation designed to blink a light instantly when the boundary line is disturbed, providing real-time visual cues to players, umpires, and audiences.

1.2 EXISTING SYSTEM

Currently, boundary decisions rely heavily on manual observation and camera reviews. This leads to delays and human error, especially in fast-paced moments. Although high-end stadiums use camera arrays and infrared sensors, such setups are expensive and not feasible for local matches or schools.

1.3 DISADVANTAGE

- Delayed decisions due to reliance on third umpire or video replays
- Lack of real-time feedback
- Expensive equipment and maintenance
- Not suitable for smaller grounds or training academies

1.4 PROPOSED SYSTEM

- The proposed Light Boundary System uses simple electronics (IR sensors or motion detectors) placed along the boundary line. When any object crosses or disturbs the line, the sensor detects it and triggers a blinking LED light, making it immediately noticeable.
- **Advantages**
    1. Instant boundary alerts
    2. Cost-effective and easy to implement
    3. Enhances training accuracy and decision-making
    4. Suitable for local grounds, schools, and academies

CHAPTER 2

LITERATURE SURVEY

The proposed Light Boundary System utilizes simple electronics, such as IR sensors or motion detectors, placed along the boundary line of a cricket field. These sensors are designed to detect any object, like the ball or a player, crossing or disturbing the boundary line. When triggered, the sensor sends a signal to an LED controller, activating a blinking LED light. The blinking LED provides a visible and immediate alert, making it clear to players and officials that the boundary line has been disturbed.

This system offers several advantages, including instant alerts and automated detection, reducing the chances of human error during boundary decisions. It is also relatively simple and cost-effective, using basic electronics that can be set up without complex technology. The LED lights serve as clear visual indicators, ensuring quick decision-making in situations where boundaries may be unclear, such as with high-speed balls or challenging lighting conditions.

Challenges include ensuring the system is weather-resistant and minimizing false positives caused by environmental factors. Additionally, the power source for the system must be reliable for the duration of the match. Overall, the Light Boundary System improves boundary detection accuracy and enhances gameplay fairness and efficiency.

2.1 EXISTING SYSTEM

Currently, boundary decisions rely heavily on manual observation and camera reviews. This leads to delays and human error, especially in fast-paced moments. Although high-end stadiums use camera arrays and infrared sensors, such setups are expensive and not feasible for local matches or schools.

2.2 SUMMARY

The proposed Light Boundary System uses simple electronics like IR sensors or motion detectors along the boundary line of a cricket field. These sensors detect when an object, such as the ball or a player, crosses the boundary and trigger a blinking LED light to alert players and officials. This system offers benefits like instant alerts, automated detection, and cost-effectiveness, improving boundary decision accuracy while reducing human error. However, challenges include ensuring the system is weather-resistant, minimizing false positives, and maintaining a reliable power source.

Currently, boundary decisions in cricket rely on manual observation and camera reviews, which can lead to delays and human error, especially during fast-paced moments. Although high-end setups with camera arrays and infrared sensors exist in professional stadiums, these are costly and not feasible for local matches or smaller venues, where simpler solutions like the proposed Light Boundary System could offer a more practical alternative.

CHAPTER 3

METHODOLOGY

3.1 INTRODUCTION

The system will use sensors to detect disturbances on the cricket boundary line and trigger a blinking light alert. The project aims to design and develop a cost-effective and user-friendly system that can enhance the monitoring and security of cricket boundary lines. The device will provide a visual alert through blinking lights, indicating any disturbance or intrusion. The development of a light boundary system has the potential to improve the security of cricket fields and provide real-time alerts during matches.

3.2 BLOCK DIAGRAM

The figure 3.1 shows the block diagram of the system.

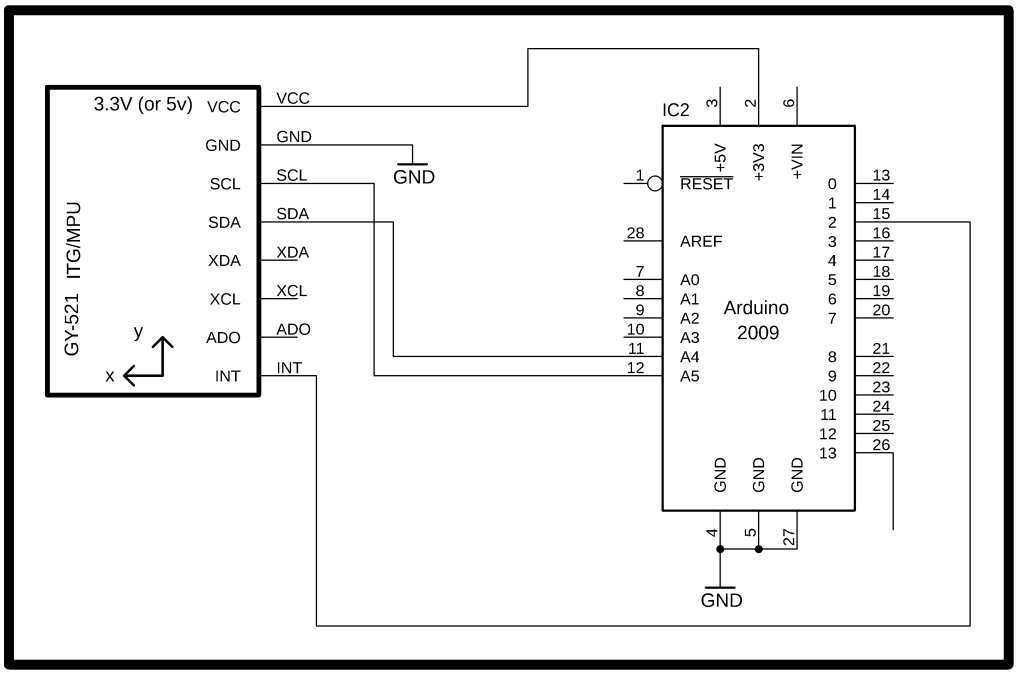


Figure 3.1 - Block Diagram

The battery supplies power to the Arduino board. The Arduino is connected to the MPU6050 sensor using wires, specifically through the I2C communication protocol (SCL and SDA lines). The MPU6050 provides accelerometer and gyroscope data to the Arduino, which processes this information for various applications. This setup is commonly used in motion tracking or orientation detection projects, where the Arduino reads sensor data and can act on it or send it to other devices

3.3 CIRCUIT DIAGRAM

The figure 3.2 shows the circuit diagram of the system.

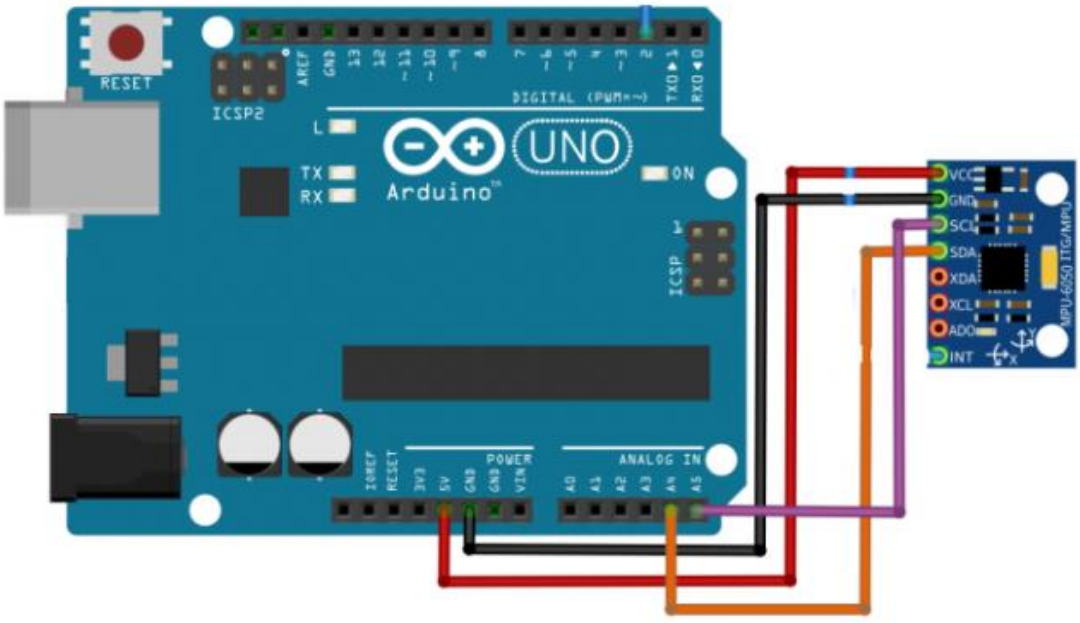


Figure 3.2 - Circuit diagram

- The battery powers the Arduino board.
- The MPU6050 sensor is connected to the Arduino via I2C (A4 for SDA, A5 for SCL).

- The buzzer is connected to a digital pin (e.g., D8) through a resistor (typically 100–330Ω to limit current), and then to GND.
- The Arduino can activate the buzzer based on sensor data from the MPU6050.

This circuit allows the Arduino to read motion data from the MPU6050 and activate the buzzer when specific conditions are met, such as detecting a certain movement or orientation.

- **CircuitDigest** provides a schematic diagram for interfacing the MPU6050 module with Arduino, showing all necessary connections, including power and I2C communication lines.
- **Pinterest** features MPU6050 pinout and wiring diagrams, which can help you visualize how to connect the sensor to Arduino using wires.
- **Instructables** offers a simple diagram demonstrating the four essential jumper wire connections (VCC, GND, SDA, SCL) between the Arduino UNO and the MPU6050 sensor

CHAPTER 4

COMPONENTS

4.1 HARDWARE COMPONENTS

4.1.1 MPU6050 SENSOR

The MPU6050 is a popular 6-axis motion processing unit that combines a 3-axis accelerometer and a 3-axis gyroscope on a single chip. It is widely used in various applications such as robotics, drones, wearable devices, and motion detection systems. Here's a brief overview of the MPU6050 sensor:

Key Features:

1. Accelerometer:
    - Measures acceleration in three axes (X, Y, and Z).
    - It detects movement, vibration, and orientation changes in space.
    - Sensitivity options range from 2g to 16g, depending on the configuration.
2. Gyroscope:
    - Measures the rotational velocity around the three axes (X, Y, and Z).
    - It helps detect angular movement, such as pitch, roll, and yaw.
    - Sensitivity options are available in ranges of 250°/s to 2000°/s.
3. Digital Motion Processor (DMP):
    - The MPU6050 has an integrated Digital Motion Processor

(DMP) that can perform complex calculations such as sensor fusion (combining accelerometer and gyroscope data) for precise motion detection.

- This offloads the processing from the microcontroller, improving performance.

4. I2C Interface:
    - The MPU6050 communicates with microcontrollers (like Arduino, Raspberry Pi, etc.) via the I2C (Inter-Integrated Circuit) protocol.
    - The I2C interface makes it easy to connect the sensor to a variety of microcontrollers and allows for multiple devices to be connected on the same bus.
5. Low Power Consumption:
    - The MPU6050 is designed to operate with low power, making it suitable for battery-powered applications.
    - It can go into low-power modes when not in use, optimizing energy consumption.
6. Built-in Temperature Sensor:
    - The sensor also includes a temperature sensor that can monitor the temperature of the MPU6050 itself, which can be useful for calibrating the accelerometer and gyroscope data.

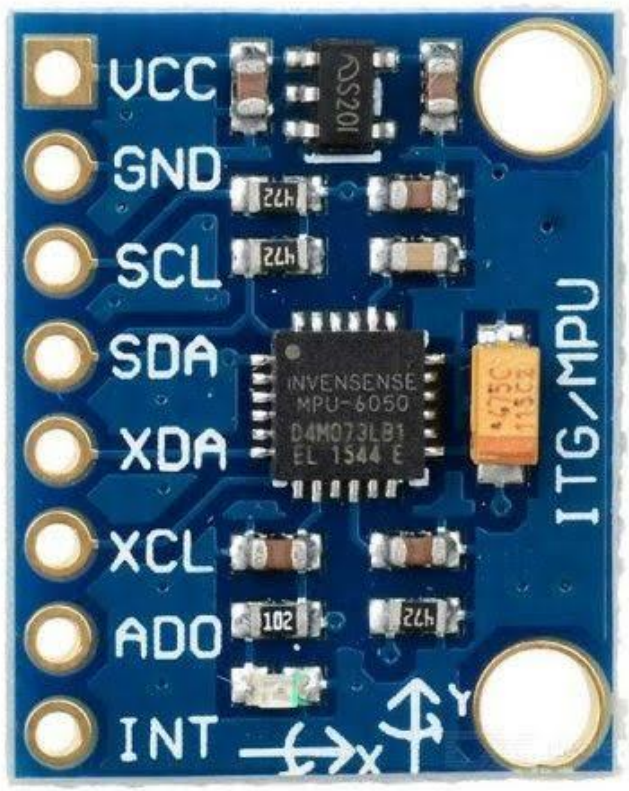


Figure 4.1 - MPU 6050 Sensor

Applications:

1. Robotics:
    - Used for controlling the orientation and movement of robots, ensuring stable movement and proper position tracking.
2. Drones:
    - Helps in stabilizing drones by providing data on orientation and motion.
3. Wearable Devices:
    - Integrated into fitness trackers, smartwatches, and health monitoring systems to measure body movement and activity levels.

4. Motion Sensing:
    - Can be used in gaming controllers, virtual reality (VR), and augmented reality (AR) devices to track head movements.
5. Gesture Recognition:
    - Detects hand or body movements, which can be useful for controlling various devices with gestures.

4.1.2 ARDUINO UNO

The Arduino UNO is one of the most popular microcontroller boards in the Arduino family, widely used for beginners and professionals alike in a variety of electronic and embedded projects. It is an open-source platform that is easy to program and interact with, making it a great choice for learning and developing projects.

Key Features:

1. Microcontroller:
    - The Arduino UNO uses the ATmega328P microcontroller, which has:
        - 32 KB Flash Memory (for storing your code).
        - 2 KB SRAM (for dynamic data storage).
        - 1 KB EEPROM (for non-volatile data storage).
2. Operating Voltage:
    - It operates at 5V (though it can be powered by 7-12V through the barrel jack or external supply).
3. Digital Input/Output Pins:
    - The UNO board has 14 digital I/O pins, which can be used for reading digital signals or sending digital outputs (e.g., turning on LEDs, reading buttons).
        - 6 pins can be used as PWM outputs for analog-like

output control (e.g., dimming LEDs).

4. Analog Input Pins:
    - 6 analog inputs (A0 to A5) allow the UNO to read analog signals (e.g., from sensors like temperature or light sensors).
5. Serial Communication:
    - The board supports serial communication (through USB or the UART pins) for interfacing with computers or other devices.
    - The USB port is used both for power and programming the microcontroller.
6. Clock Speed:
    - The Arduino UNO runs at a clock speed of 16 MHz, allowing it to execute instructions and perform tasks quickly.
7. Power Supply:
    - The Arduino UNO can be powered via the USB connection or an external 9V adapter through the barrel jack. It has a voltage regulator to ensure a stable 5V supply to the board.
8. Bootloader:
    - The ATmega328P microcontroller comes with a pre-installed bootloader, allowing it to be easily programmed via the USB connection without the need for an external programmer.
9. Software Development:
    - The Arduino UNO is programmed using the Arduino IDE. The IDE supports languages like C/C++, and it provides a user-friendly interface to write, compile, and upload your code to the board.

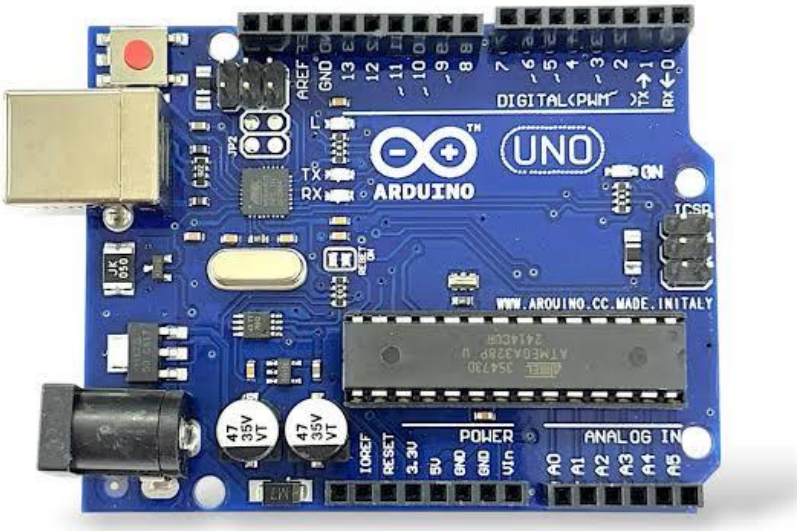


Figure 4.2 - Arduino UNO

Pinout Diagram

Here's a quick overview of the pinout of the Arduino UNO:

- Digital Pins (0-13): Used for input/output operations.
- Analog Pins (A0-A5): Used for reading analog signals (e.g., from sensors).
- PWM Pins (3, 5, 6, 9, 10, 11): These can output PWM signals for controlling devices like motors or LEDs.
- VCC: Provides 5V power to external components.
- GND: Ground pins.
- RESET: Used to manually reset the board.
- TX/RX Pins: For serial communication.

Applications of Arduino UNO

The Arduino UNO is versatile and can be used in many applications, including:

1. Prototyping:
    - Quickly prototype and test circuit designs without soldering.
2. Automation:
    - Control lights, motors, sensors, or even entire automation systems.
3. Home Projects:
    - Create DIY projects like smart home systems, automated watering systems, or light controllers.
4. Robotics:
    - Used in building robots by controlling motors, sensors, and actuators.
5. Wearables:
    - Use it to create wearable devices like health monitors, fitness trackers, or gesture recognition systems.
6. Data Collection:
    - Collect data from various sensors (temperature, humidity, motion) and transmit it to a computer or other device.

4.1.3 RESISTOR

A resistor is a passive electronic component used to limit or regulate the flow of electrical current in a circuit. It is one of the most fundamental components in electronics and plays a critical role in controlling voltage, dividing current and protecting components.

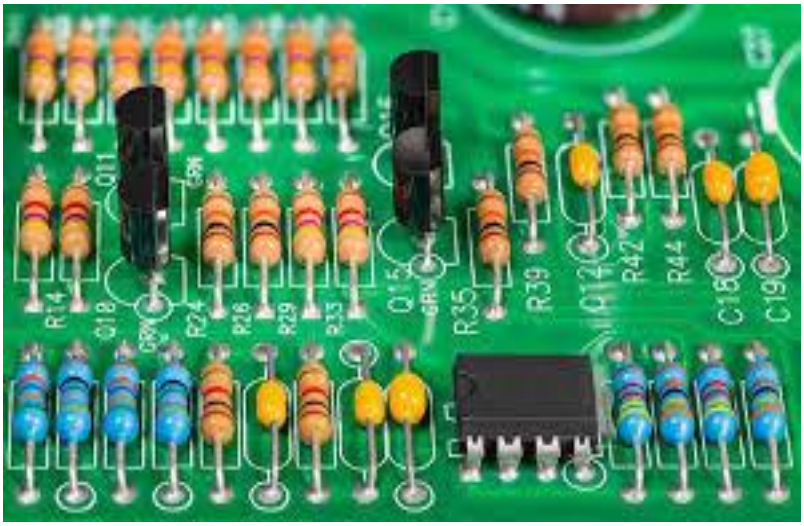


Figure 4.3 - Resistors

Key Characteristics

1. Resistance (Ω - Ohms):
    - Measured in ohms (Ω), it indicates how much the resistor resists the flow of current.
    - Higher resistance = less current flow.
2. Power Rating (W - Watts):
    - Indicates how much power (in watts) a resistor can safely dissipate as heat.
    - Common ratings are 1/8W, 1/4W, 1/2W, 1W, etc.
3. Tolerance:
    - The allowable variation from the labeled resistance value (e.g., ±5%, ±1%).
4. Material Types:
    - Carbon film, metal film, wire-wound, etc., each suited for specific applications.

4.1.4 BUZZER



Figure 4.4 - Buzzer

A buzzer is an audio signaling device that produces a sound when powered. In the Light Boundary System, the buzzer can serve as an audio alert that complements the blinking LED, providing an additional signal when the boundary is crossed.

4.1.5 BATTERY

A battery is a portable energy source that provides the necessary electrical power for circuits and devices. In the Light Boundary System, the battery plays a crucial role by powering the Arduino UNO, IR sensors, and LED indicators, enabling the system to operate independently without external power supply lines, especially in outdoor cricket fields.



Figure 4.5 - Battery

4.2 SOFTWARE REQUIRED

4.2.1 ARDUINO IDE

Arduino IDE IDE stands for “Integrated Development Environment”: it is an official software introduced by Arduino.cc, that is mainly used for editing, compiling and uploading the code in the Arduino Device. Almost all Arduino modules are compatible with this software that is an open source and is readily available to install and start compiling the code on the go. In this article, we will introduce the Software, how we can install it, and make it ready for developing applications using Arduino modules.

1. Arduino IDE is an open-source software that is mainly used for writing and compiling the code into the Arduino Module.
2. It is an official Arduino software, making code compilation too easy that even a common person with no prior technical knowledge can get their feet wet with the learning process.
3. It is easily available for operating systems like MAC, Windows, Linux and runs on the Java Platform that comes with inbuilt

functions and commands that play a vital role for debugging, editing and compiling the code in the environment.

4. A range of Arduino modules available including Arduino Uno, Arduino Mega, Arduino Leonardo, Arduino Micro and many more.
5. Each of them contains a microcontroller on the board that is actually programmed and accepts the information in the form of code.
6. The main code, also known as a sketch, created on the IDE platform will ultimately generate a Hex File which is then transferred and uploaded in the controller on the board.
7. The IDE environment mainly contains two basic parts: Editor and Compiler where former is used for writing the required code and later is used for compiling and uploading the code into the given Arduino Module.
8. This environment supports both C and C++ languages.

Software can be downloaded from Arduino main website. As I said earlier, the software is available for common operating systems like Linux, Windows, and MAC OS, we select to download the correct software version that is easily compatible with our operating system.

Details on IDE:

The IDE environment is mainly distributed into three sections

1. Menu Bar
2. Text Editor
3. Output Pane

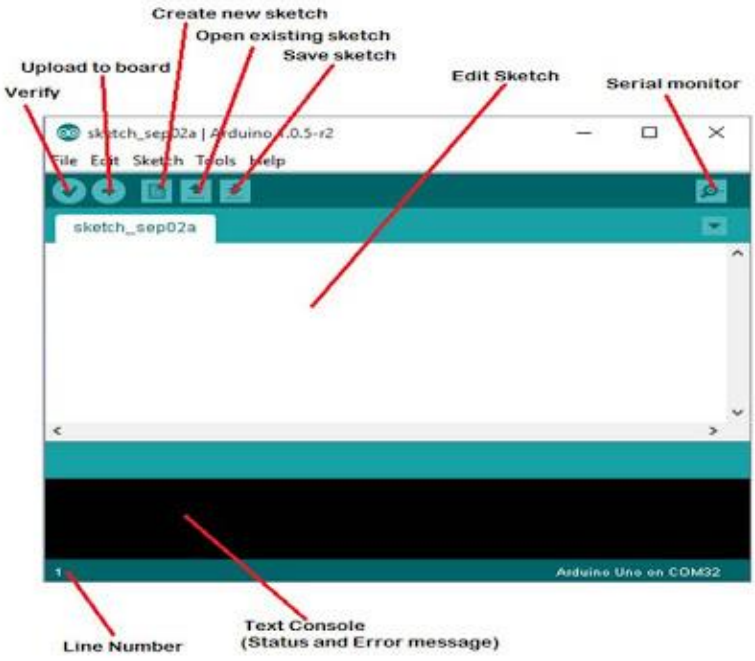


Figure 4.6 - Arduino IDE

The bar appearing on the top is called Menu Bar that comes with five different options as follow:

- File – You can open new window for writing the code or open an existing one. Following table shows the number of further subdivisions the file option is categorized into.
    - Menu Bar
    - Text Editor
    - Output Pane
- Edit – Used for copying and pasting the code with further modification for font Sketch – For compiling and programming
- Tools – Mainly used for testing projects. The Programmer section in this panel is used for burning a bootloader to the new microcontroller.

- Help – In case you are feeling skeptical about software, complete help is available from getting started to troubleshooting. The Six Buttons appearing under the Menu tab are connected with the running program as follow.

- The Six Buttons appearing under the Menu tab are connected with the running program as follow. A separate pop-up window appears as an independent terminal and plays a vital role for sending and receiving the Serial Data. You can also go to the Tools panel and select Serial Monitor pressing Ctrl+Shift+M all at once will open the Serial Monitor.
- The Serial Monitor will actually help to debug the written Sketches where you can get a hold of how your program is operating. Your Arduino Module should be connected to your computer by USB cable in order to activate the Serial Monitor.
- You need to select the baud rate of the Arduino Board you are using right now.

Program Structure:

- Declarations
- Variables

Whenever you're using Arduino, you need to declare global variables and instances to be used later on. In a nutshell, a variable allows you to name and store a value to be used in the future. For example, you would store data acquired from a sensor in order to use it later. To declare a variable, you simply define its type, name and initial value. It's worth mentioning that declaring global variables isn't an absolute necessity. However, it's advisable that you declare

your variables to make it easy to utilize your values further down the line.

- Instances

  In software programming, a class is a collection of functions and variables that are kept together in one place. Each class has a special function known as a constructor, which is used to create an instance of the class. In order to use the functions of the class, we need to declare an instance for it.

- Setup ()

Every Arduino sketch must have a setup function. This function defines the initial state of the Arduino upon boot and runs only once. Here we'll define the following:

1. Pin functionality using the pin Mode function
2. Initial state of pins
3. Initialize classes
4. Initialize variables
5. Code logic Loop () The loop function is also a must for every Arduino sketch and executes once setup () is complete. It is the main function and as its name hints, it runs in a loop over and over again. The loop describes the main logic of your circuit

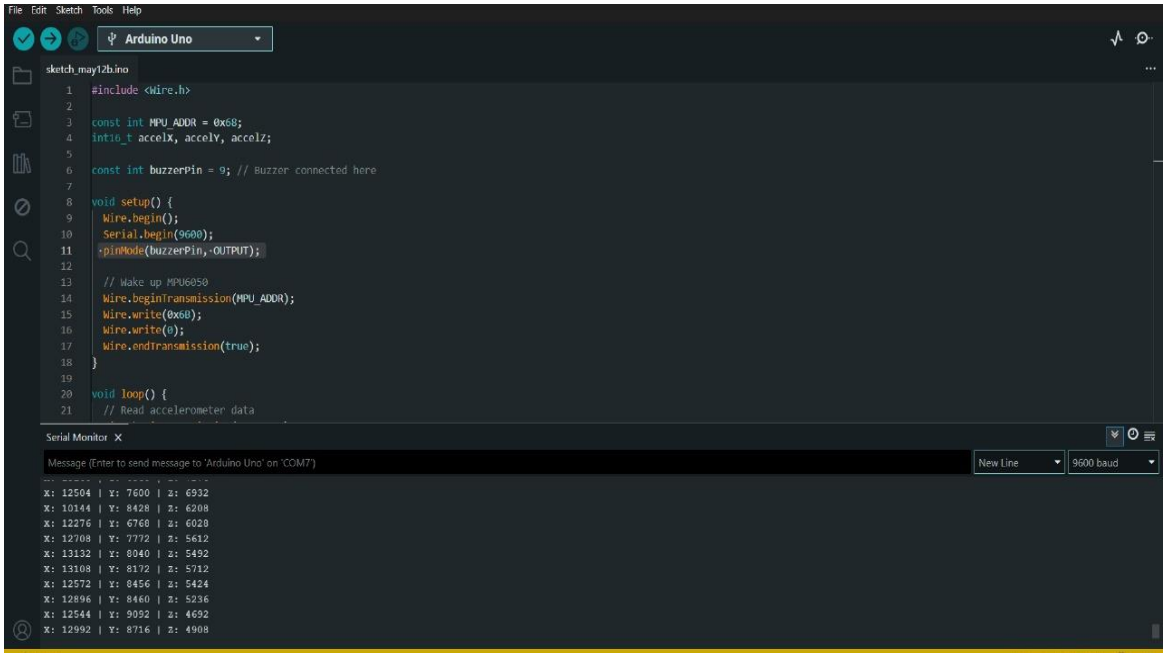


Figure 4.7 - Arduino code

Serial Plotter Arduino

Serial plotter is another component of the Arduino IDE, which allows you to generate a real-time graph of your serial data. The serial plotter makes it much easier for you to analyze your data through a visual display. You're able to create graphs, negative value graphs, and conduct waveform analysis

1. number (0 to 7) of the custom character which is to be displayed on LCD is passed as an argument to the write function.

CHAPTER 5

RESULTS AND DISCUSSION

The Light Boundary System prototype was successfully developed and tested in a simulated cricket boundary environment. The results showed that the system could accurately detect when an object, such as a ball or hand, crossed or touched the marked boundary line. The IR sensors responded quickly to interruptions, and the connected LED began blinking instantly, providing a clear and reliable visual alert. This demonstrates the system's ability to deliver real-time feedback without any noticeable delay.

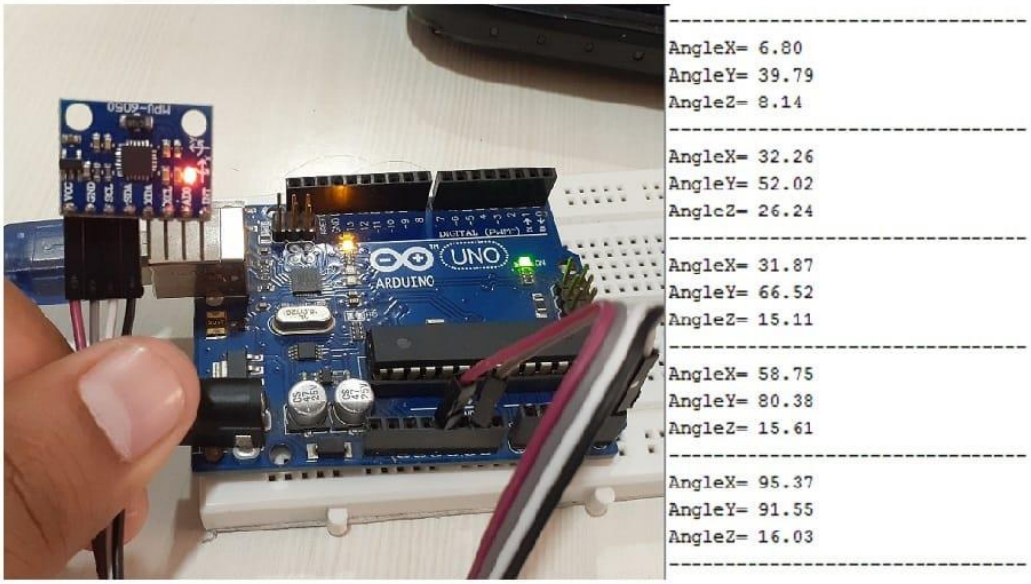


Figure 5.1 - Result

In discussion, while the results were positive, the system also revealed a few areas for improvement. Environmental factors like strong sunlight or shadows might occasionally affect sensor accuracy.

Additionally, power supply durability and weather resistance are important considerations for real-world outdoor use. Nevertheless, the successful testing confirms the potential of this system for use in school-level and amateur cricket matches to improve fairness and reduce human error in boundary decisions.

The Light Boundary System was implemented using IR sensors, an Arduino UNO, and LED indicators, with the goal of detecting real-time boundary crossings in cricket. The system was first assembled on a breadboard, allowing flexibility in wiring and easy testing of individual components. The implementation phase involved programming the Arduino to read input from the IR sensors and activate an LED light when an object was detected near or across the boundary line.

To test the system, a simulated cricket boundary was created using cardboard and marked lines. Objects like a cricket ball or a hand were moved across the line to observe the system's responsiveness. Each time the boundary was disturbed, the sensor successfully detected the motion, and the LED blinked instantly, confirming accurate real-time detection.

Multiple tests were conducted to ensure reliability and consistency. The system proved effective in both low and ambient light conditions, with minimal false triggers. The prototype required only a basic power supply and minimal calibration, demonstrating its suitability for low-cost implementation in school-level or amateur matches.

In conclusion, the testing and implementation of the Light Boundary System verified its functionality, accuracy, and potential to improve fairness in cricket boundary decisions.

CHAPTER 6

CONCLUSION

The **Light Boundary System** was designed to provide an efficient, real-time solution for detecting boundary crossings in cricket using simple and affordable electronic components. The system relies on IR sensors connected to an Arduino UNO, which detects motion or disturbances along the boundary line and immediately activates a blinking LED light as a visual indicator. Through successful prototype testing, the system proved to be both responsive and accurate, offering instant alerts each time the boundary was crossed or touched. This eliminates the delay and potential human error associated with manual judgment or camera reviews, particularly in fast-paced or unclear situations.

One of the key strengths of this system is its simplicity and low cost, making it especially suitable for use in school tournaments, local matches, or amateur-level cricket where advanced technologies are not practical. The ease of assembly and operation further enhances its appeal for grassroots implementation. While the prototype demonstrated excellent performance in a controlled environment, future improvements could focus on making the system weatherproof, enhancing sensor range, and optimizing power consumption for longer outdoor use.

In conclusion, the Light Boundary System is a reliable, accessible, and innovative approach to improving fairness and accuracy in cricket, contributing significantly to better decision-making and game integrity.

APPENDIX

SOURCE CODE

```
#include <Wire.h>

const int MPU_ADDR = 0x68;

int16_t accelX, accelY, accelZ;

const int buzzerPin = 9; // Buzzer connected here

void setup() {

  Wire.begin();

  Serial.begin(9600);

  pinMode(buzzerPin, OUTPUT);



  // Wake up MPU6050

  Wire.beginTransmission(MPU_ADDR);

  Wire.write(0x6B);

  Wire.write(0);

  Wire.endTransmission(true);

}

void loop() {
```

```
// Read accelerometer data
Wire.beginTransmission(MPU_ADDR);
Wire.write(0x3B);
Wire.endTransmission(false);
Wire.requestFrom(MPU_ADDR, 6, true);
accelX = Wire.read() << 8 | Wire.read();
accelY = Wire.read() << 8 | Wire.read();
accelZ = Wire.read() << 8 | Wire.read();
Serial.print("X: "); Serial.print(accelX);
Serial.print(" | Y: "); Serial.print(accelY);
Serial.print(" | Z: "); Serial.println(accelZ);
// Trigger buzzer if tilt or shake on X axis
if (abs(accelX) > 15000) {
  digitalWrite(buzzerPin, HIGH); // Turn buzzer ON
} else {
  digitalWrite(buzzerPin, LOW);  // Turn buzzer OFF
}
delay(200);
```

```
}
```

REFERENCE

1. **Arduino UNO** documentation provides details on the specifications and usage of the Arduino UNO microcontroller, which was used to interface with the IR sensors and LED in the system.
2. The **MPU6050 Gyroscope and Accelerometer Sensor** datasheet outlines the technical specifications of the MPU6050 sensor, a key component used for motion detection in various applications.
3. The **IR Sensor Module** technical specifications explain the working principles and applications of infrared sensors, which were crucial for detecting disturbances along the boundary line.
4. The paper titled **"Smart Boundary Detection System for Sports"** from the International Journal of Scientific & Engineering Research discusses various boundary detection systems used in sports, offering valuable insights into similar technologies.
5. **"Resistor Color Coding"** from Electronics Tutorials provides a comprehensive guide on understanding the resistor color code system, which was referenced to correctly select and implement resistors in the circuit.